

Research Platform

A beginner-friendly explanation of the system, why it exists, and its live build status

Generated from the repository documentation and live workspace on 16 August 2026.

One-sentence summary: this project will take one big research question, split it into smaller questions, research those pieces in parallel, assemble a report, and then verify every factual claim against the source that supposedly supports it.

1. The problem this project solves

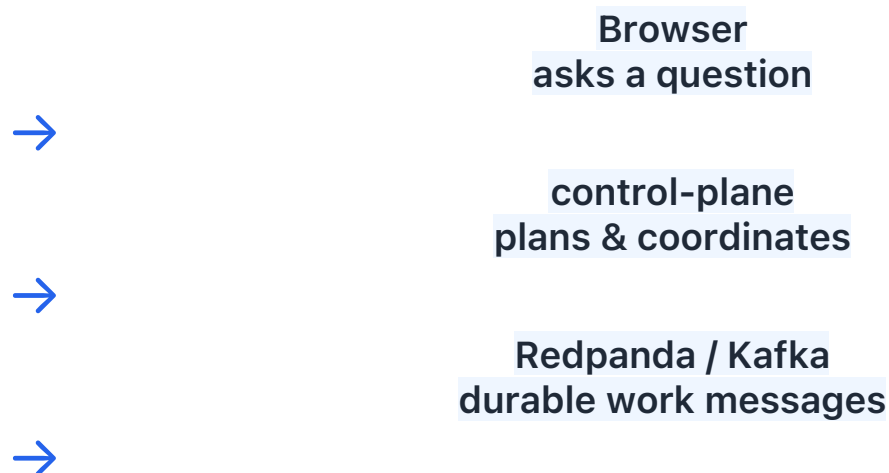
Imagine asking: *"Should India invest more in renewable energy?"* A normal chatbot may write a convincing answer, but it can invent figures, mix up sources, or attach a citation that does not actually support the sentence next to it.

This platform treats research as a workflow rather than one giant AI prompt:

- 1 Break the broad question into smaller, independent sub-questions.
- 2 Search trusted sources and collect clean text.
- 3 Let several researcher workers work in parallel.
- 4 Build a report from structured claims instead of untraceable prose.
- 5 Send every factual claim back through a critic that checks its original source and stores the supporting passage.

The last step is the special part. A report is not considered trustworthy merely because it has footnotes. Each factual statement must be checkable against specific evidence.

2. The intended architecture



agent-service
researcher / writer / critic



retrieval-service
search, fetch, cache, rate-limit
retrieval-service



SearXNG
web search



extractor sidecar
clean article text
control-plane
run state / fan-in



Postgres + pgvector
retrieval cache / locks / limits



Redis

Why three services?

Part

Plain-English job

Why it is separate

Port

retrieval-service

The system's librarian and courier: searches, fetches pages, cleans content, caches answers, and protects external websites/search budgets.

It is the single "quota boundary." No AI worker gets to call a search engine directly.

8081

agent-service

The AI workforce: researchers investigate, writers assemble reports, critics verify claims.

The AI roles share the same operational needs, so they are Spring profiles in one app rather than three extra deployments.

8082

control-plane

The front desk and coordinator: receives the question, plans work, sends tasks out, waits for results, and streams progress to the browser.

Only this service should be externally reachable in the future.

8083

common

A small shared Java library for common data records such as future claims and findings.

It contains data shapes only—no Spring app, database, Kafka client, or business logic.

Not runnable

3. Key technology choices, explained simply

Tool

What it is

Why this project uses it

Java 21 + Spring Boot

The main backend language and web framework.

Matches the author's Java/Spring background while adding modern Java features such as records and virtual threads.

Maven multi-module build

One parent project that builds several related modules together.

Lets all services share dependency versions and compile as one coherent project while remaining separately runnable.

Docker Compose

A local "start the whole supporting stack" file.

Starts Postgres, Redis, Redpanda, and SearXNG together without installing each one manually.

Postgres + pgvector

A reliable relational database; pgvector later adds vector similarity search.

Holds durable research-run state. A restart does not forget how much work is finished.

Redis

Very fast in-memory key/value storage.

Will hold disposable shared state: cached searches, rate-limit counters, and short-lived locks.

Redpanda

A Kafka-compatible event broker.

Moves long-running research tasks durably between services. If a worker restarts, work is not lost.

SearXNG

A self-hosted search aggregator.

Queries multiple search engines and returns one JSON response, avoiding a separate paid API integration for each provider.

Flyway

A database change-history tool.

Applies numbered SQL migrations in order. The schema is explicit,

reviewable SQL—not Hibernate guessing at tables.

4. How a completed research run will work

- 1 **Question arrives:** the browser sends one broad question to `control-plane`.
- 2 **Planning:** an AI planner turns it into roughly 6–8 flat sub-questions. “Flat” means none depends on another this month, so all can start immediately.
- 3 **Fan-out:** `control-plane` publishes the sub-tasks to `research.subtasks` in Redpanda/Kafka.
- 4 **Research:** researcher workers ask `retrieval-service` to search and fetch sources. They do not bypass it.
- 5 **Fan-in:** every completed sub-task atomically increments a Postgres counter. Exactly one worker observes “all expected tasks are done” and triggers the writer.
- 6 **Writing:** the writer creates structured claims. A claim has text plus one source ID, rather than loose prose with vaguely related footnotes.
- 7 **Verification:** the critic re-fetches the cited source, finds the best evidence passage, and marks a claim supported, partial, unsupported, contradicted, or unreachable.
- 8 **Presentation:** the browser receives live progress through SSE and eventually sees a report with visible verification badges.

The fan-in decision: why Postgres instead of a Java counter?

An in-memory `AtomicInteger` is like a tally counter kept in one person’s pocket. If that person restarts, it is gone. If two copies of the service run, each copy has a different tally. Instead, Postgres owns the tally:

```
UPDATE dag_levels
SET completed = completed + 1
WHERE run_id = ? AND level = 0
RETURNING completed, expected;
```

The database makes the increment atomic. One and only one completion sees the final count. Because it is in Postgres, the count survives an application restart.

5. What is deliberately not being built

Good scope is a feature. The project intentionally postpones or excludes:

- DAG dependencies between sub-questions. The database reserves fields for them, but every current task stays at level 0.
- Contradiction reconciliation, multi-provider search routing, rich UI, multi-user billing, and early CI/CD/VM deployment.

- Kubernetes, service mesh, API gateway, Terraform, and extra services during the core month.

A separate optional Week 5 later introduces k3s, KEDA autoscaling for the bursty agent service, ingress, and JWT protection—but only after the local version works.

6. Learning value and interview stories

This is intentionally a learning project, not just an app. Each hand-built piece gives a concrete answer to an interview question.

Topic

Practical story this project will provide

Cache-aside

Check Redis → on a miss call the external system → store JSON with a TTL. Redis being down makes requests slower, not incorrect.

Redis Lua token bucket

A per-domain rate limiter must read and update atomically. A Lua script avoids two requests both passing a check-then-act race.

Cache stampede protection

When many agents miss the same cache key, one wins a short Redis lock while others wait and reread. This is the “thundering herd” problem.

Parallel Java I/O

CompletableFuture plus an explicit/virtual-thread executor fetches independent URLs in parallel without letting one dead URL sink the batch.

Distributed coordination

Postgres UPDATE ... RETURNING, not a ConcurrentHashMap, coordinates completion across restarts and replicas.

Verification/evaluation

Deliberately corrupt claims and measure whether the critic catches them. This demonstrates AI reliability rather than merely generating prose.

7. What is built right now

The following is based on the live workspace inspected for this guide.

Area

Current state

Project skeleton

Built. Root Maven aggregator contains `common`, `retrieval-service`, `agent-service`, and `control-plane`.

Local infrastructure

Built. Compose defines Postgres/pgvector, Redis, Redpanda, and SearXNG. SearXNG JSON output was enabled.

Database baseline

Built. Flyway migration creates `runs`, `dag_nodes`, and `dag_levels`; Flyway also creates its own history table.

Kafka topics

Created locally. `research.subtasks` has 12 partitions, `research.findings` has 6, and `agent.events` has 3.

Service health

Verified. Apps are configured for 8081/8082/8083 and Actuator health checks have been verified.

retrieval API contract

Built as stubs. Java records define search/extract/quota payloads. The controller exposes all three planned endpoints and currently returns placeholders.

Source tiering

Partially built and tested. YAML lists tier-1/2 domains and tier-4 patterns. `SourceTierResolver` maps URL hosts to a tier and has five focused unit tests. It is not connected to live SearXNG result mapping yet.

Real SearXNG call

Not built. `POST /api/v1/search` still returns an empty list; the next implementation step is a `RestClient`-based SearXNG client.

Extractor sidecar

Not built. The extract endpoint is intentionally a stub until the Python/trafilatura sidecar exists.

Agents, orchestration, SSE, verification

Not built. These start in later weeks once retrieval is real.

8. The database tables that already exist

Table

Purpose

Important constraints

`runs`

One record for each top-level research question.

`id` is a UUID primary key; `question/status/timestamps` cannot be null; `report` is optional until writing finishes.

`dag_nodes`

One record for each sub-question/research task.

`run_id` is a required foreign key to `runs`; `depends_on` is a nullable self-reference reserved for later; `level` defaults to 0.

`dag_levels`

The durable completion counter for a run level.

Composite primary key (`run_id`, `level`) guarantees one counter per level per run; `expected`, `completed`, and `deadline` make fan-in/release decisions possible.

flyway_schema_history

Flyway's own bookkeeping table.

Records that version 1 ran, so it will never run the same migration twice.

9. The retrieval service, in detail

This is the active area of development. It owns all outbound web access, because allowing each AI worker to make its own searches would make budget, caching, and politeness impossible to control centrally.

Endpoint

Input

Current result

Eventually

POST /api/v1/search

A list of queries, result limit, freshness, and minimum tier.

Returns empty results, zero spent credits, zero cache hits.

Call SearXNG, map its JSON into internal results, assign source tiers, normalize/cache requests, and limit results.

POST /api/v1/extract

A list of URLs.

Returns empty documents.

Call the Python extractor; return clean text, tier, and a status such as OK or UNREACHABLE.

GET /api/v1/quota

No input.

Returns placeholder `creditsRemaining: 1000`.

Return the project's actual self-imposed budget/use information.

The API uses Java **records**, a concise immutable data-carrier form. For example, `SearchResult(url, title, snippet, tier)` automatically has fields, a constructor, read accessors, equality, and a useful `toString`—without Lombok or handwritten boilerplate.

Source quality tiers

The current YAML configuration classifies sources before agents use them:

- **Tier 1:** exact primary/official sources currently including PIB, RBI, and ISRO.
- **Tier 2:** exact reputable journalism domains currently including The Hindu, Indian Express, and NDTV.
- **Tier 3:** the safe default for any unknown domain.
- **Tier 4:** pattern-matched low-confidence sources, currently Blogspot and WordPress-hosted blogs.

The resolver strips `www.`, lowercases a URL's host, checks the exact lists,

then checks the glob patterns. Its tests cover a tier-1 domain, a tier-2 www URL, Blogspot, WordPress, and an unknown default.

10. Important conventions and guardrails

- **Migration first, entity second:** write reviewed Flyway SQL before creating JPA entities. Hibernate validates rather than silently edits production-like schemas.
- **Constructor injection only:** dependencies arrive in constructors, not hidden field injection.
- **Records rather than Lombok:** DTOs and agent contracts are immutable, explicit Java data types.
- **HTTP for short lookups; Kafka for durable work:** services call retrieval over HTTP, but researcher/writer hops use Kafka because they can take 5–90 seconds and must survive restarts.
- **Version cache keys:** use prefixes such as `search:v1:`. When cache semantics change, bump the version instead of trying to delete every old key.
- **Tests should prefer Testcontainers:** later integration tests should use real disposable dependencies instead of mocks that pretend Redis/Kafka/Postgres behave more simply than they do.

11. Known traps and live documentation discrepancies

Current Maven compiler setting: the live root `pom.xml` inspected for this guide contains `<java.version>21</java.version>`, but does not currently show `<maven.compiler.release>${java.version}</maven.compiler.release>`. The session log says that property was added earlier to fix “records are not supported in source 8.” Before relying on Java records through Maven, verify/restore that property and reload Maven in IntelliJ.

Spring Boot version mismatch: the live Maven BOM pins Spring Boot 4.1.0. Some narrative instruction files still say Spring Boot 3.3.x and Spring AI 1.0. Treat the live `pom.xml` as the build truth; the narrative stack line needs later cleanup.

- In Spring Boot 4.x, `flyway-core` alone did not trigger Flyway auto-configuration. The fixed dependency is `spring-boot-starter-flyway`, with the PostgreSQL Flyway module retained.
- SearXNG serves HTML by default. Its mounted configuration must explicitly allow JSON.
- The SearXNG container owns port 8080. The three Spring apps must use 8081, 8082, and 8083.

- IntelliJ can retain stale Maven/classpath state. After dependency or compiler-property changes, use *Reload All Maven Projects*; delegating build/run actions to Maven avoids an IDE-vs-terminal mismatch.
- Postgres-backed tests require Docker Desktop and the Compose Postgres container to be running.

12. Roadmap from here

Near-term next step

Why it comes next

Connect `POST /api/v1/search` to SearXNG using Spring `RestClient`.

It replaces the verified HTTP stub with a real search result while keeping the controller thin.

Connect the existing source-tier resolver to those mapped results.

Every result should carry a useful tier before agents decide what to read.

Design and implement query/URL normalization and versioned cache keys.

These make identical requests map to one predictable cache entry; this is intentionally a hands-on learning task.

Build Python/FastAPI/trafilatura extractor sidecar.

Only then can `/extract` return real cleaned article text.

Add manual Redis cache-aside, token bucket, single-flight lock, parallel fetching, and extractor semaphore.

This is the scheduled Redis/concurrency learning work—do not hide it behind framework shortcuts.

Implement researcher fan-out/fan-in, then structured writer and critic verification.

These are the core multi-agent and reliability features.

13. Definition of success

By the end, a person should be able to clone the project, start the local stack, submit a question, watch several researchers work, read a report, and click a claim to see the exact passage that supports or contradicts it. The strongest demo is not “the AI wrote a report.” It is: **“the system can prove where each factual sentence came from, and it catches an intentionally injected fabrication.”**

Source material reviewed: AGENTS.md, CLAUDE.md, docs/PLAN.md, docs/LEARNING.md, all module README files, standard Spring Initializr HELP files, and current live source/configuration files.